Session 16 Homework — Authorization That Isn't a Role Check

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

By the end of class, real, workspace-scoped, role-based authorization gates every real write route on Projects and Workspaces — your own shared guard and decorator proved that in teamwork. Issues are still only protected by identity (`JwtAuthGuard`) — any signed-in user, even a total stranger to a workspace, can still create, edit, or delete any Issue anywhere in this app. This gap is deliberately not closed the same way. `AUTH-AND-PERMISSIONS.md` is explicit: an Issue may be edited or deleted only by its **assignee**, its **reporter**, or a workspace **Admin/Owner** — this is a genuinely different kind of rule than "does this user hold this role," and nobody has shown you, live or in teamwork, how to build this kind.

Goal

Creating an Issue requires the creator to already be a real member of its own target workspace. Updating or deleting an Issue requires the requester to be that specific issue's own assignee, its own reporter, or a workspace Admin/Owner — proven correct for a real member who is none of those three things and is still, correctly, rejected.

What to build

Research, then build:

- **Two new real fields on Issue**: `reporterId` (who created it — set automatically, server-side, from the real authenticated request; never accepted from the client) and `assigneeId` (who it's currently assigned to — nullable, optionally supplied by the client on create or update). Find their exact expected shape in this project's own domain documentation.
- **A membership check on Issue creation.** An Issue's own workspace is not directly on the Issue — it has to be resolved through the Issue's own `projectId`, to the Project's own `workspaceId`, to a real `WorkspaceMember` lookup. Research how to navigate a two-level parent relationship like this from inside a Nest service (hint: you already have the services for both hops available to inject).
- **A genuinely different authorization shape for update and delete** — not a guard, not a decorator, THIS time. A guard COULD technically be built for this (it could load the Issue itself, resolve its Project, resolve its WorkspaceMember, and authorize, exactly like teamwork's own `WorkspaceRoleGuard` already does) — but think about the cost: the service method is about to load that same Issue anyway, to perform the mutation. Build this check where the resource naturally already lives: the service method itself, right before the mutation happens, so the Issue is loaded once, not twice.
- **A real design question to resolve yourselves:** should the ownership check look at BOTH assignee and reporter, or only one? Should a workspace Admin/Owner always be allowed regardless of either? Resolve this from `AUTH-AND-PERMISSIONS.md` directly — it is explicit; don't guess.

CORE (required)

- `Issue.reporterId` (required, server-set only) and `Issue.assigneeId` (nullable) exist and are wired through creation and update correctly.
- Creating an Issue requires the real authenticated creator to already be a member (any role) of the Issue's own target workspace — resolved through the Issue's own Project. A real outsider must be rejected.
- Updating and deleting an Issue requires the requester to be that specific issue's own real assignee, its own real reporter, or a workspace Admin/Owner — resolved and checked in the service layer, not a guard. Prove the one case that makes this rule genuinely different from a role check: a real workspace member who is neither the assignee, the reporter, nor an Admin/Owner on THIS specific issue must still be rejected, even though they really are a member of the workspace.
- No new HTTP endpoint beyond what already exists on Issues — this is authorization logic inside the existing `create` / `update` / `remove` methods, not a new route.

STRETCH (required)

An issue's own `assigneeId`, whenever supplied (on creation or on reassignment), must itself refer to a real member of the issue's own workspace — reuse the identical membership-checking capability CORE already built, applied to a second field instead of the requester's own id. Prove it rejects assigning an issue to someone outside its own workspace, on both creation and update.

CHALLENGE (required)

Write short, direct, written answers, placed directly above the new ownership-check logic you built in CORE. This is reasoning about a real design choice, not a search for a technical limitation:

1. Could this ownership rule (assignee/reporter/Admin-Owner) technically be implemented in a Guard? Explain how.
2. Why did this application choose service-layer authorization for Issue update/delete instead?
3. What duplicate work might happen if a Guard loaded the Issue and the service then loaded it again?
4. When does metadata-driven route authorization (`@RequireWorkspaceRole()`) fit better?
5. When does resource-instance/domain authorization fit naturally beside service logic instead?
6. How could a larger application avoid duplicate loading while still using a Guard for this? Reason only — do not implement a new architecture.
7. Why must the authorization decision use CURRENT Issue/WorkspaceMember state rather than claims copied into the JWT?

Rules

Allowed: Mongoose schema/service edits (already used throughout this course), `@CurrentUser()` (already used repeatedly this session), service-layer logic calling another service's own existing public methods, everything already shown in every prior session. No new packages.

Not yet: no `@RequireWorkspaceRole()` / `WorkspaceRoleGuard` use anywhere on Issues — this homework's whole point is that this specific rule needs a different shape. No invite/add-member endpoint. No changes to Projects' or Workspaces' own authorization, already finished in teamwork.

Test it

Never assume any user's own id, workspace id, or issue id is any particular number. Capture every real id from the real responses every time, and use those captured values — never a literal number — in every proof below.

BASH

```
# register/log in a real outsider (no membership anywhere), and a real workspace member

# outsider cannot create an issue in a workspace they do not belong to
curl -i -X POST http://localhost:3000/issues -H "Authorization: Bearer $TOKEN_OUTSIDER" \
  -H "Content-Type: application/json" -d "{\"projectId\":$PROJECT_ID,\"title\":\"Rogue\"}"

# a real member creates an issue -- they become its real reporter
ISSUE=$(curl -s -X POST http://localhost:3000/issues -H "Authorization: Bearer
$TOKEN_MEMBER" \
  -H "Content-Type: application/json" -d "{\"projectId\":$PROJECT_ID,\"title\":\"Real
issue\"}")
echo $ISSUE

# a DIFFERENT real member of the SAME workspace, with no relationship to THIS issue, is
still rejected
curl -i -X PATCH http://localhost:3000/issues/$ISSUE_ID -H "Authorization: Bearer
$TOKEN_UNRELATED_MEMBER" \
  -H "Content-Type: application/json" -d '{"status":"in_progress"}'

# the real reporter can edit their own issue
curl -i -X PATCH http://localhost:3000/issues/$ISSUE_ID -H "Authorization: Bearer
$TOKEN_MEMBER" \
  -H "Content-Type: application/json" -d '{"status":"in_progress"}'
```

Design your own equivalent proof for STRETCH: attempt to assign (or reassign) an issue to a real user who is NOT a member of its own workspace, on both creation and update, and confirm both are rejected.

Done when

- `Issue.reporterId` / `assigneeId` exist with the correct shape, and `reporterId` can never be supplied by a client.
- A real outsider cannot create an issue in a workspace they do not belong to; a real member can, and becomes its real reporter.
- A real workspace member who is neither this specific issue's own assignee, reporter, nor an Admin/Owner is correctly rejected on update/delete — proven, not merely implemented.

- The real reporter, the real assignee (once one exists), and a real Admin/Owner can each correctly mutate the issue.
- STRETCH: assigning an issue to a non-member is rejected on both creation and reassignment.
- Your CHALLENGE answers are grounded in this app's own real fields and real gaps, not a generic explanation of ownership-based access control.

Working time

About 90–110 minutes — this introduces a genuinely different authorization shape from teamwork's own guard-based mechanism; budget real time to understand WHY it's different before implementing it, not only to type the code.